Laboratorul 10. Funcții și excepții

Apărarea: detecția unui atac brute force din auth.log

fișă de laborator · 2 × 90 min (cu pauză la mijloc) · două lucrări · se lucrează individual, la terminal

> **CE FACEȚI ASTĂZI.** Treceți în scaunul apărării. Dintr-un jurnal auth.log de pe o mașină atacată, scrieți parse_auth.py care scoate, robust, eșecurile de autentificare și le numără pe adrese (Lucrarea A). Apoi construiți un detector de brute force cu prag reglabil, validare la intrare și raport JSON (Lucrarea B). Fiecare pas folosește o funcție care ÎNTOARCE un rezultat, nu una care doar tipărește.

> **CUM LUCRAȚI (deep-end).** Fișa dă obiectivul, scheletul cu # TODO și comanda de rulare, nu rândul gata scris. După fiecare pas rulați și comparați cu caseta verde „Pe ecran trebuie să vezi". Explicația de programare (funcții, parametri, scope, try/except, raise) e în suportul de curs; aici o puneți în practică.

Fișierul de start

Din secțiunea sesiunii de pe ELSE descărcați start/auth.log și puneți-l în depozitul de curs, sub arsenal/aparare/start/. Conține un jurnal ssh de pe o mașină de laborator: printre rânduri normale, un atac brute force de la o adresă și câteva rânduri stricate, intenționat.

> **CONTEXT ȘI ETICĂ.** Jurnalul e de la o mașină izolată, de laborator. Analiza de jurnale se face pe sisteme care îți aparțin sau pe care ești autorizat. Detecția e partea de apărare a aceleiași tehnici pe care ai folosit-o ca atacator; scopul e să înțelegi ambele scaune.

LUCRAREA A (nucleu). parse_auth.py: scoate și numără eșecurile

Obiectiv: arsenal/aparare/parse_auth.py citește jurnalul, scoate eșecurile de autentificare într-o listă de fișe și le numără pe adrese. Robust: un rând stricat nu oprește parcurgerea.

A1. Scheletul și funcția care întoarce o fișă (20 min)

Un rând de eșec arată așa: Failed password for root from 203.0.113.9 port 51230 ssh2. Scrie o funcție esec(linie) care întoarce (user, ip) dacă rândul e un eșec, altfel None. Folosești o expresie regulată dată:

```
import re
TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\S+) port")

def esec(linie):
    m = TIPAR.search(linie)
    # TODO: daca m e None, intoarce None;
    #       altfel intoarce (m.group(1), m.group(2))
    ...

print(esec("... Failed password for root from 203.0.113.9 port 51230 ssh2"))
print(esec("<linie stricata fara structura>"))
```

Pe ecran trebuie să vezi:

```
('root', '203.0.113.9')
None
```

De observat. Funcția întoarce None pentru un rând care nu e un eșec. None nu e o eroare, e un semnal: „aici nu e nimic de raportat". Cel care cheamă funcția decide ce face cu el.

A2. Lista de eșecuri, robustă la rânduri stricate (20 min)

citeste_esecuri(cale) parcurge fișierul și strânge fișele (user, ip) într-o listă de dicționare. Sare peste rândurile care nu sunt eșecuri. Pune citirea fișierului într-un try/except, ca un fișier lipsă să dea un mesaj clar, nu un traceback.

```
def citeste_esecuri(cale):
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
        print("nu gasesc", cale)
        return esecuri
    with f:
        for linie in f:
            r = esec(linie)
            # TODO: daca r nu e None, adauga {"user": r[0], "ip": r[1]}
    return esecuri

esecuri = citeste_esecuri("arsenal/aparare/start/auth.log")
print("esecuri:", len(esecuri))
```

Pe ecran trebuie să vezi:

```
esecuri: 15
```

A3. Numărarea pe IP: o funcție cu return (18 min)

Numeri de câte ori apare fiecare adresă. Poți cu un dicionar și .get, sau cu Counter din collections, care face exact asta. Funcția ÎNTOARCE numărătoarea, nu o tipărește.

```
from collections import Counter

def pe_ip(esecuri):
    # TODO: intoarce un Counter al adreselor din esecuri
    #       (Counter(e["ip"] for e in esecuri))
    ...

counts = pe_ip(esecuri)
for ip, n in counts.most_common():
    print(f"  {ip}: {n}")
```

Pe ecran trebuie să vezi:

```
  203.0.113.9: 13
  198.51.100.7: 2
```

De observat. O adresă iese mult peste rest: 13 eșecuri față de 2. Asta e semnătura unui brute force. most_common() sortează de la mare la mic, deci atacatorul apare primul.

A4. Rezumatul (12 min)

O funcție main() leagă totul: citește, numără, tipărește rezumatul. Rulezi din rădăcina depozitului.

```
python -m arsenal.aparare.parse_auth arsenal/aparare/start/auth.log
```

Pe ecran trebuie să vezi:

```
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

> **DE REȚINUT.** Fiecare funcție întoarce un rezultat (fișa, lista, numărătoarea). Le legi ca pe o conductă: ieșirea uneia intră în următoarea. O funcție care doar tipărea ar fi rupt lanțul.

LUCRAREA B (extindere). Detectorul de brute force

Obiectiv: arsenal/aparare/detector.py decide ce adrese sunt suspecte, cu un prag reglabil, validare la intrare și raport JSON. Refolosești funcțiile din parse_auth printr-un import.

B1. suspecti() cu valoare implicită și validare (20 min)

O funcție suspecti(counts, prag=10) întoarce adresele cu cel puțin prag eșecuri, sortate descrescător. Pragul are o valoare implicită, dar îl poți schimba. Și un prag absurd (sub 1) nu are sens: îl oprești cu raise.

```
def suspecti(counts, prag=10):
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")
    # TODO: intoarce lista de (ip, n) cu n >= prag,
    #       sortata descrescator dupa n
    ...

from collections import Counter
c = Counter({"203.0.113.9": 13, "198.51.100.7": 2})
print(suspecti(c))
print(suspecti(c, prag=2))
```

Pe ecran trebuie să vezi:

```
[('203.0.113.9', 13)]
[('203.0.113.9', 13), ('198.51.100.7', 2)]
```

De observat. Cu pragul implicit 10, doar atacatorul trece. Cu prag 2, apare și a doua adresă. Pragul separă zgomotul (o parolă greșită) de semnal (un atac). De aceea e un parametru.

B2. Validarea, verificată (8 min)

Verifică faptul că un prag absurd e oprit cu un mesaj clar:

```
python -c "from arsenal.aparare.detector import suspecti; suspecti({}, 0)"
```

Pe ecran trebuie să vezi:

```
ValueError: prag invalid: 0 (cel putin 1)
```

De observat. raise oprește funcția la intrare, cu o explicație. E mai bine decât să continue și să marcheze orice adresă ca suspectă, în tăcere.

B3. argparse și raportul JSON (22 min)

main() primește jurnalul, un --prag (implicit 10, type=int) și un --json. Leagă funcțiile din parse_auth cu suspecti și scoate fie un rezumat citibil, fie un raport JSON.

```
import argparse, json
from collections import Counter
from arsenal.aparare.parse_auth import citeste_esecuri

def main():
    p = argparse.ArgumentParser()
    p.add_argument("log")
    p.add_argument("--prag", type=int, default=10)
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    esecuri = citeste_esecuri(a.log)
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, a.prag)
    # TODO: cu --json, scoate json.dumps cu {esecuri, pe_ip, suspecti};
    #       fara, un rezumat cu "BRUTE FORCE" pe fiecare suspect
```

```
    ...
python -m arsenal.aparare.detector arsenal/aparare/start/auth.log
```

Pe ecran trebuie să vezi:

```
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
Suspecti (prag 10):
  203.0.113.9  (13 incercari)  BRUTE FORCE
```

B4. Un test care prinde regresia (15 min)

Scrie arsenal/aparare/test_detector.py cu două teste cu date scrise de mână: numărarea e corectă și pragul taie unde trebuie. Rulează fără jurnal real.

```
from collections import Counter
from arsenal.aparare.detector import suspecti

def test_pragul_taie():
    c = Counter({"a": 13, "b": 2})
    assert suspecti(c, 10) == [("a", 13)]

def test_prag_invalid():
    import pytest
    with pytest.raises(ValueError):
        suspecti({}, 0)
```

```
pytest -q arsenal/aparare/test_detector.py
```

Pe ecran trebuie să vezi:

```
2 passed
```

OPȚIONAL (pentru vârful sălii sau acasă)

1. --fereastra secunde: numără doar eșecurile dintr-un interval de timp (atacul e dens în timp). Parsezi ora din rând și compari.

2. parse_auth și pe un jurnal real: pe mașina ta de laborator, /var/log/auth.log (ai nevoie de sudo pentru a-l citi). Compari forma rândurilor cu capturile.

3. --top N: scoate doar primele N adrese după numărul de eșecuri, cu un comprehension peste most_common.

Ce predați și cum se notează

Predarea e prin git, în depozitul tehnici-prog-<nume>. Nimic pe e-mail, nicio captură de ecran.

```
git add arsenal/aparare
git commit -m "aparare: parse_auth robust + detector brute force"
git push
```

Livrabil	Ce conține
arsenal/aparare/parse_auth.py	funcții care ÎNTORC (esec, citeste_esecuri, pe_ip), robuste la rânduri stricate (Lab A).
arsenal/aparare/detector.py	suspecti cu valoare implicită și raise, argparse, raport JSON (Lab B).
arsenal/aparare/test_detector.py	două teste care trec cu pytest (Lab B).

Se punctează: funcții cu return (nu print), care nu crapă pe input stricat (try/except pe nume), un parametru cu valoare implicită și o validare cu raise. Asistentul vă poate cere să explicați de ce o funcție întoarce None și unde ați pune un prag.